

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN **MEDICAL DIARY**

Môn học: CSC10014 - Tư duy tính toán

Giáo viên hướng dẫn: ThS. Võ Nam Thực Doan
ThS. Lê Nhật Nam
ThS. Ngô Ngọc Đăng Khoa

Nhóm thực hiện: Nhóm 3 – Lớp 24CTT4

Thành viên: 24120394 – Nguyễn Đặng Khôi Nguyên
24120085 – Lê Nguyễn Thùy Linh
24120370 – Trần Thị Lợi
24120474 – Trịnh Vỹ Triết
24120480 – Cái Lâm Trường

TP. Hồ Chí Minh, 2026

Mục lục

1	Giới thiệu đề tài	3
1.1	Tên đề tài	3
1.2	Bối cảnh	3
1.3	Mục tiêu	4
2	Áp dụng tư duy tính toán	5
2.1	Phân rã vấn đề (Decomposition)	5
2.1.1	Từ bài toán gốc đến bài toán rõ ràng	5
2.1.2	Đầu vào, đầu ra, và giới hạn	6
2.1.3	Các module chức năng hệ thống	7
2.2	Nhận diện mẫu (Pattern Recognition):	7
2.2.1	Vấn đề phân tán thông tin thực tế và Mẫu quản lý dữ liệu tập trung (CRUD Pattern)	7
2.2.2	Tiếp cận theo hoàn cảnh và Mẫu phân cấp dữ liệu (Context-Driven Pattern)	8
2.2.3	Thói quen sinh hoạt định kỳ và Mẫu lập lịch theo chu kỳ (Schedule Pattern)	8
2.2.4	Mối quan hệ thời điểm và Mẫu ủy quyền ngắn hạn (Temporary Delegation Pattern)	8
2.3	Trừu tượng hóa (Abstraction)	8
2.3.1	Trừu tượng hóa Đối tượng Thông tin (Information Object Abstraction)	9
2.3.2	Trừu tượng hóa Cơ chế Vận hành (Operational Abstraction)	9
2.4	Thiết kế thuật toán (Algorithm Design)	10
2.4.1	Thuật toán quét nhắc nhở uống thuốc tự động (Timezone-Aware Reminder Job)	10
2.4.2	Thuật toán đánh giá quyền truy cập thông tin bệnh nhân (Dynamic Authorizer)	11
2.4.3	Thuật toán truy xuất khẩn cấp qua mã QR (SOS Emergency Access)	12
3	Thiết kế hệ thống	14
3.1	Tổng quan Công cụ Sử dụng	14
3.2	Thiết kế Chi tiết Frontend	16
3.2.1	Cấu trúc thư mục Frontend	16
3.2.2	Luồng tương tác Frontend	17
3.3	Thiết kế Chi tiết Backend	17
3.3.1	Cấu trúc thư mục Backend	17
3.3.2	Cấu trúc phân tầng trong từng Module	18

3.4	Thiết kế Cơ sở dữ liệu và Cơ chế Tự động hóa	18
3.4.1	Sơ đồ luồng hoạt động tự động hóa	18
3.5	Kiến trúc Triển khai (Deployment Architecture)	19
4	Thiết kế dữ liệu	20
5	Kết quả đạt được.....	22
5.1	Các chức năng đã triển khai	22
5.2	Giao diện chính.....	22
6	Đánh giá và hướng phát triển.....	23
6.1	Ưu điểm.....	23
6.2	Hạn chế	23
6.3	Hướng phát triển	23
7	Kết luận	24

Chương 1

Giới thiệu đề tài

1.1 Tên đề tài

Medical Diary – Hệ thống lưu trữ, quản lý sức khỏe tích hợp.

1.2 Bối cảnh

Trong bối cảnh chuyển đổi số trong lĩnh vực y tế ngày càng phát triển, nhu cầu quản lý và lưu trữ thông tin sức khỏe cá nhân một cách tập trung đang trở nên cần thiết hơn bao giờ hết. Hiện nay, nhiều người vẫn lưu trữ các thông tin y tế quan trọng như bệnh nền, lịch sử khám bệnh, kết quả xét nghiệm, đơn thuốc, dị ứng thuốc và lịch tiêm chủng dưới dạng giấy tờ hoặc phân tán ở nhiều cơ sở y tế khác nhau. Điều này gây khó khăn cho việc theo dõi tình trạng sức khỏe lâu dài cũng như tra cứu thông tin khi cần thiết.

Bên cạnh đó, các bệnh nhân mắc bệnh mãn tính hoặc phải sử dụng thuốc thường xuyên cần theo dõi lịch dùng thuốc, tái khám định kỳ và những thay đổi trong tình trạng sức khỏe hằng ngày. Tuy nhiên, việc ghi chép thủ công hoặc ghi nhớ bằng cá nhân dễ dẫn đến tình trạng quên thuốc, bỏ lỡ lịch tái khám hoặc thất lạc thông tin y tế quan trọng.

Đặc biệt, trong các tình huống khẩn cấp như tai nạn giao thông, đột quỵ, ngất xỉu hoặc mất ý thức tại nơi công cộng, người gặp nạn thường không thể cung cấp các thông tin y tế cần thiết cho nhân viên cấp cứu. Việc thiếu dữ liệu như bệnh nền, dị ứng thuốc, nhóm máu, thuốc đang sử dụng hoặc thông tin người thân cần liên hệ có thể làm chậm quá trình đánh giá ban đầu và ảnh hưởng đến hiệu quả xử lý trong “thời gian vàng” cấp cứu.

Ngoài ra, hiện nay vẫn chưa có nhiều giải pháp phổ biến cho phép người dùng chủ động quản lý hồ sơ sức khỏe cá nhân và chia sẻ nhanh các thông tin cần thiết trong những trường hợp đặc biệt. Điều này tạo ra nhu cầu về một hệ thống hỗ trợ lưu trữ, theo dõi và cung cấp thông tin y tế một cách thuận tiện, an toàn và dễ tiếp cận.

Xuất phát từ thực tế đó, dự án Medical Diary được xây dựng nhằm cung cấp một nền tảng quản lý sức khỏe cá nhân toàn diện. Hệ thống cho phép người dùng lưu trữ hồ sơ y tế, quản lý thuốc, theo dõi lịch sử điều trị, thiết lập nhắc nhở sức khỏe và truy cập nhanh các thông tin quan trọng thông qua chế độ SOS trong trường hợp khẩn cấp.

Qua đó, Medical Diary góp phần hỗ trợ người dùng chủ động hơn trong việc chăm sóc sức khỏe bản thân, đồng thời giúp việc tiếp cận thông tin y tế trở nên nhanh chóng và hiệu quả hơn khi cần thiết.

Medical Diary được xây dựng nhằm hỗ trợ người dùng quản lý thông tin thuốc, lịch dùng thuốc và các dữ liệu sức khỏe cá nhân một cách tập trung và thuận tiện.

1.3 Mục tiêu

- Quản lý và lưu trữ hồ sơ sức khỏe cá nhân, nhật ký sức khỏe và các thông tin liên quan đến tình trạng của người dùng.
- Quản lý danh sách thuốc, thiết lập lịch nhắc nhở và theo dõi lịch sử sử dụng thuốc.
- Hỗ trợ lưu trữ và cung cấp nhanh thông tin y tế cần thiết trong các tình huống khẩn cấp.
- Hỗ trợ theo dõi, lưu trữ và quản lý các chỉ số sức khỏe như nhịp tim, huyết áp, nhịp thở và các chỉ số liên quan khác.
- Hỗ trợ bác sĩ tiếp cận hồ sơ sức khỏe, theo dõi tình trạng bệnh nhân và hỗ trợ quá trình điều trị.
- Đảm bảo an toàn dữ liệu, phân quyền chặt chẽ

Chương 2

Áp dụng tư duy tính toán

Tư duy tính toán được áp dụng thông qua bốn thành phần chính: Phân rã vấn đề, Nhận dạng mẫu, Trừu tượng hóa và Thiết kế thuật toán. Bốn thành phần trên được áp dụng xuyên suốt từ giai đoạn phân tích và thiết kế, triển khai hệ thống

2.1 Phân rã vấn đề (Decomposition)

2.1.1 Từ bài toán gốc đến bài toán rõ ràng

Để giải quyết bài toán quản lý và khai thác dữ liệu sức khỏe cá nhân, hệ thống Medical Diary được phân rã thành các nhóm vấn đề nhỏ hơn.

Trước hết, thông tin y tế của người dùng hiện đang phân tán ở nhiều nguồn khác nhau như hồ sơ giấy, kết quả xét nghiệm, đơn thuốc, ứng dụng của cơ sở y tế hoặc trí nhớ cá nhân. Do đó, cần xây dựng một cơ chế lưu trữ tập trung để người dùng có thể quản lý hồ sơ sức khỏe, lịch sử điều trị, thuốc đang sử dụng và các thông tin liên quan trên cùng một hệ thống.

Tiếp theo, nhu cầu sử dụng dữ liệu sức khỏe thay đổi tùy theo từng ngữ cảnh. Trong tình huống cấp cứu, nhân viên y tế hoặc người hỗ trợ chỉ cần tiếp cận nhanh các thông tin quan trọng như nhóm máu, dị ứng, bệnh nền, thuốc đang sử dụng và thông tin liên hệ khẩn cấp. Trong khi đó, khi thăm khám thông thường, bác sĩ cần truy cập thêm dữ liệu lịch sử sức khỏe, chỉ số theo dõi dài hạn và nhật ký sức khỏe để hỗ trợ chẩn đoán. Vì vậy, dữ liệu cần được phân tầng và cung cấp theo từng mục đích sử dụng khác nhau.

Ngoài ra, hệ thống phải đảm bảo cân bằng giữa khả năng tiếp cận thông tin và quyền riêng tư của người dùng. Các thông tin thiết yếu phục vụ cấp cứu phải được truy cập nhanh chóng, trong khi những dữ liệu nhạy cảm hơn chỉ được chia sẻ khi có sự cho phép của chủ sở hữu dữ liệu. Đối với bác sĩ, quyền truy cập vào hồ sơ sức khỏe cần được kiểm soát, giới hạn theo thời gian và phạm vi được người dùng cho phép.

Từ việc phân rã bài toán trên, có thể xác định các thành phần chính của hệ thống bao gồm dữ liệu đầu vào, dữ liệu đầu ra và các ràng buộc cần được đảm bảo trong quá trình thiết kế và vận hành hệ thống.

2.1.2 Đầu vào, đầu ra, và giới hạn

Đầu vào (Input):

- Thông tin tài khoản và hồ sơ cá nhân của người dùng.
- Thông tin thuốc: tên thuốc, liều lượng, tần suất và thời gian sử dụng.
- Thông tin hồ sơ sức khỏe: bệnh nền, dị ứng, nhóm máu và tiền sử bệnh.
- Thông tin liên hệ khẩn cấp.
- Nhật ký sức khỏe và các triệu chứng do người dùng ghi nhận.
- Các chỉ số sức khỏe được nhập thủ công hoặc thu thập từ thiết bị hỗ trợ (nếu có).
- Hồ sơ, tài liệu y tế do người dùng tải lên (đơn thuốc, kết quả xét nghiệm, hình ảnh y tế, v.v.).

Đầu ra (Output):

- Thông báo nhắc nhở uống thuốc.
- Nhật ký sử dụng thuốc và lịch sử điều trị.
- Hồ sơ sức khỏe cá nhân.
- Lịch sử điều trị và khám bệnh.
- Thông tin phục vụ tình huống khẩn cấp.
- Lịch sử theo dõi các chỉ số sức khỏe.
- Danh sách thuốc đang sử dụng và đơn thuốc từ bác sĩ.
- Thống kê và biểu đồ sức khỏe theo thời gian.

Các ràng buộc, giới hạn:

- Thông tin sức khỏe cá nhân phải được lưu trữ an toàn và chỉ người dùng được cấp quyền mới có thể truy cập.
- Dữ liệu của mỗi người dùng phải được tách biệt, đảm bảo người dùng không thể truy cập dữ liệu của người khác.
- Hệ thống nhắc nhở sử dụng thuốc phải hoạt động đúng thời gian đã được thiết lập.
- Thông tin khẩn cấp phải có thể được truy cập nhanh chóng không cần đăng nhập trong các tình huống cần thiết.
- Dữ liệu sức khỏe phải được đồng bộ và cập nhật chính xác trên toàn hệ thống.

- Hệ thống phải có khả năng mở rộng để bổ sung các chức năng quản lý sức khỏe mới trong tương lai.
- Hệ thống yêu cầu kết nối Internet để đồng bộ dữ liệu và sử dụng đầy đủ các chức năng.

2.1.3 Các module chức năng hệ thống

Dựa trên việc xác định luồng dữ liệu vào/ra và các ràng buộc đi kèm, bài toán lớn tiếp tục được phân rã thành các khối chức năng lõi (modules) độc lập về mặt nghiệp vụ nhưng có khả năng tương tác chặt chẽ với nhau:

- **Authentication Module:** Quản lý đăng nhập, đăng ký, xin quyền, cấp quyền và phê duyệt bảo mật tài khoản.
- **Profiles Module:** Quản lý thông tin cá nhân và hồ sơ sức khỏe nền tảng. Hỗ trợ cơ chế tìm kiếm và cấp quyền truy cập hồ sơ y tế tạm thời cho bác sĩ.
- **Health Data Module:** Quản lý và lưu trữ thông tin các chỉ số sức khỏe sinh học, ghi nhật ký hàng ngày, quản lý đơn thuốc nâng cao và lưu trữ bệnh án cá nhân.
- **Emergency Module:** Module chuyên biệt quản lý và điều phối luồng truy cập thông tin khẩn cấp (SOS) một cách tối giản.
- **Notification Module:** Chịu trách nhiệm quản lý hệ thống hàng đợi thông báo, lập lịch và nhắc nhở thời gian thực việc sử dụng thuốc của người dùng.

2.2 Nhận diện mẫu (Pattern Recognition):

Từ việc phân rã bài toán quản lý dữ liệu sức khỏe cá nhân, chúng ta nhận thấy các vấn đề thực tế trong đời sống và các khuôn mẫu cấu trúc trong hệ thống có sự tương thích và giao thoa chặt chẽ với nhau. Quá trình nhận dạng mẫu được thể hiện qua các điểm cốt lõi sau:

2.2.1 Vấn đề phân tán thông tin thực tế và Mẫu quản lý dữ liệu tập trung (CRUD Pattern)

Thực tế: Thói quen sinh hoạt khiến người dùng lưu trữ thông tin y tế một cách rời rạc dựa trên loại tài liệu (đơn thuốc kẹp trong sổ giấy, kết quả xét nghiệm lưu ở tin nhắn, tiền sử bệnh chỉ nhớ trong đầu). Tuy nhiên, bản chất hành vi của người dùng đối với mọi loại giấy tờ này đều lặp lại: họ cần tạo mới khi đi khám, đọc lại khi cần xem, chỉnh sửa khi có thay đổi và xóa bỏ khi không còn giá trị.

Mẫu hệ thống: Hệ thống nhận diện được sự trùng khớp giữa hành vi thực tế này với mô hình **CRUD Pattern** (Create - Read - Update - Delete). Việc nhận dạng mẫu giúp quy nạp toàn bộ các thực thể đời sống (Hồ sơ, Thuốc, Liên hệ khẩn cấp) về một khuôn

mẫu quản lý dữ liệu đồng nhất.

2.2.2 Tiếp cận theo hoàn cảnh và Mẫu phân cấp dữ liệu (Context-Driven Pattern)

Thực tế: Hành vi khai thác thông tin y tế của con người thay đổi đột ngột dựa trên hoàn cảnh thực tế. Trong tình huống cấp cứu (SOS), người hỗ trợ cần tiếp cận thông tin sinh mạng ngay lập tức mà không thể chờ đợi xác thực; ngược lại, khi thăm khám thông thường, bác sĩ cần xem thông tin chi tiết nhưng phải đảm bảo sự riêng tư của bệnh nhân.

Mẫu hệ thống: Hệ thống nhận diện được tính chất đối lập này và phân loại thành hai khuôn mẫu xử lý: Mẫu tương tác nhanh (Fast-track) dành cho các tình huống khẩn cấp, tối giản và Mẫu tương tác bảo mật (Secure-track) dành cho các bối cảnh thông thường, chuyên sâu.

2.2.3 Thói quen sinh hoạt định kỳ và Mẫu lập lịch theo chu kỳ (Schedule Pattern)

Thực tế: Việc chăm sóc sức khỏe của con người không diễn ra ngẫu nhiên mà luôn lặp đi lặp lại theo chu kỳ đời sống hoặc phác đồ điều trị (uống thuốc đúng giờ mỗi ngày, tái khám định kỳ sau vài tháng, đo huyết áp mỗi tuần). Con người rất dễ quên các hành động mang tính lặp lại này do nhịp sống bận rộn.

Mẫu hệ thống: Hệ thống nhận diện tất cả các hành vi thực tế trên đều thuộc về Schedule Pattern (Mẫu lập lịch). Dù là nhắc uống thuốc hay nhắc tái khám, chúng đều có chung một quy luật vận hành là kích hoạt dựa trên mốc thời gian lặp lại.

2.2.4 Mối quan hệ thời điểm và Mẫu ủy quyền ngắn hạn (Temporary Delegation Pattern)

Thực tế: Trong đời sống, mối quan hệ giữa bệnh nhân và bác sĩ mang tính thời điểm. Người dùng có nhu cầu chia sẻ thông tin sức khỏe cho bác sĩ trong suốt đợt điều trị tại bệnh viện, nhưng muốn mối liên kết này tự động chấm dứt khi đợt điều trị kết thúc để bảo vệ quyền riêng tư cá nhân.

Mẫu hệ thống: Bài toán tâm lý và mối quan hệ xã hội này trùng khớp với mẫu Ủy quyền có thời hạn. Hệ thống nhận diện khuôn mẫu này để chuẩn bị cho việc thiết kế các liên kết dữ liệu có thể tự động đóng lại theo thời gian thực tế.

2.3 Trừu tượng hóa (Abstraction)

Trong Tư duy Tính toán (Computational Thinking), trừu tượng hóa là quá trình chọn lọc các đặc tính cốt lõi của bài toán thực tế và lọc bỏ đi những chi tiết dư thừa hoặc không có giá trị quyết định đến việc giải quyết vấn đề. Đối với hệ thống **Medical Diary**, tư duy trừu tượng hóa được áp dụng ở cả hai bình diện: trừu tượng hóa đối

tượng thông tin và trừu tượng hóa cơ chế vận hành. Điều này giúp tối giản hóa mức độ phức tạp của bài toán quản lý y tế cá nhân trong thế giới thực thành các mô hình khái niệm có thể xử lý và lập trình được.

2.3.1 Trừu tượng hóa Đối tượng Thông tin (Information Object Abstraction)

Trong bối cảnh y tế và chăm sóc sức khỏe, thông tin về con người, thuốc men và bệnh án rất đa dạng và phức tạp. Để mô hình hóa chúng thành dữ liệu số, hệ thống đã thực hiện các bước trừu tượng hóa khái niệm:

- **Trừu tượng hóa thực thể người dùng (Bệnh nhân/Bác sĩ):** Một con người vật lý ngoài đời thực với vô số đặc điểm sinh hoạt, nghề nghiệp, ngoại hình được hệ thống tinh giản thành một tập hợp các thuộc tính y tế thiết yếu như nhóm máu, tiền sử dị ứng, thông số sinh mạng thời gian thực, và các cài đặt bảo mật. Các thông tin định danh cá nhân nhạy cảm khác chỉ được xem xét dưới dạng các chuỗi dữ liệu đã được mã hóa an toàn, loại bỏ hoàn toàn các chi tiết hành vi sinh hoạt không liên quan.
- **Trừu tượng hóa thực thể thuốc và đơn thuốc:** Thay vì quan tâm đến giá cả, mẫu mã bao bì, nhà sản xuất hay hình dáng vật lý của thuốc, hệ thống chỉ tập trung vào các thông tin cốt lõi cấu thành nên phác đồ điều trị: tên hoạt chất/biệt dược, liều lượng sử dụng, tần suất dùng và tổng số ngày điều trị theo chỉ định của bác sĩ chuyên khoa.
- **Trừu tượng hóa chỉ số sinh học:** Các chỉ số sinh mạng trong thực tế là những đại lượng vật lý biến thiên liên tục theo thời gian (như nhịp tim, nhiệt độ cơ thể). Hệ thống trừu tượng hóa chúng thành các tập hợp điểm dữ liệu rời rạc có gắn nhãn thời gian cụ thể (time-stamps) để phục vụ cho các thuật toán phân tích xu hướng và vẽ biểu đồ.
- **Trừu tượng hóa liên kết y khoa (Quyền truy cập):** Mối quan hệ phức tạp và nhạy cảm giữa bệnh nhân và bác sĩ ngoài đời thực được trừu tượng hóa thành cơ chế phân quyền (Consent Permission). Quyền này được biểu diễn như một sự ủy quyền có giới hạn chặt chẽ về mặt thời gian và phạm vi dữ liệu được phép đọc, có khả năng tự động hết hạn hoặc thu hồi bất kỳ lúc nào.

2.3.2 Trừu tượng hóa Cơ chế Vận hành (Operational Abstraction)

Bên cạnh thực thể dữ liệu, các hành vi tương tác thực tế của con người cũng được hệ thống trừu tượng hóa thành các mô hình vận hành và biểu thức logic:

1. **Mô hình hóa thói quen dùng thuốc (Adherence State Machine):** Hành động uống thuốc phức tạp trong đời thực (bao gồm các tình huống như uống chậm

trễ, quên liều, làm rơi thuốc, uống không đủ liều...) được trừu tượng hóa thành một máy trạng thái nhị phân tối giản: Đã uống (Taken) hoặc Bỏ lỡ (Missed). Mọi biến số ngoại cảnh đều bị loại bỏ, chỉ giữ lại mốc thời gian thực tế ghi nhận nhằm tính toán tỷ lệ tuân thủ điều trị của bệnh nhân.

2. **Tự động hóa lịch trình nhắc nhở (Temporal Trigger):** Khả năng ghi nhớ lịch sinh hoạt của con người vốn dễ bị xao nhãng bởi cuộc sống thường nhật được trừu tượng hóa thành cơ chế kiểm tra ngầm (Background Job) chạy theo chu kỳ thời gian thực. Cơ chế này hoạt động độc lập với ngữ cảnh sinh hoạt của người dùng, liên tục đối chiếu sự trùng khớp giữa thời gian hệ thống và thời gian nhắc nhở đã thiết lập để kích hoạt các sự kiện thông báo tương ứng.
3. **Cơ chế cổng thông tin cứu sinh (Emergency SOS Gateway):** Trong các tình huống khẩn cấp, quy trình thăm khám và xác thực rườm rà ngoài đời thực được rút gọn tối đa thành một cổng truy cập tối giản. Hệ thống trừu tượng hóa toàn bộ hồ sơ bệnh án dày cộm thành một thẻ định danh sinh mạng kỹ thuật số tối giản, chỉ trích xuất duy nhất các thông tin có thể cứu sống người dùng (như nhóm máu, dị ứng thuốc đặc biệt) dựa trên cài đặt bảo mật mà không yêu cầu các bước đăng nhập hay xác minh mật khẩu của bệnh nhân.

2.4 Thiết kế thuật toán (Algorithm Design)

Để đảm bảo hiệu năng và tính đúng đắn của các giải pháp trừu tượng hóa nêu trên, hệ thống thiết kế các thuật toán vận hành chi tiết:

2.4.1 Thuật toán quét nhắc nhở uống thuốc tự động (Timezone-Aware Reminder Job)

Thuật toán này chạy định kỳ mỗi 5 phút qua hệ thống lập lịch cơ sở dữ liệu (pg_cron), thực hiện gọi HTTP POST đến FastAPI backend qua pg_net. Thuật toán giải quyết vấn đề lệch múi giờ giữa máy chủ (UTC) và múi giờ sinh hoạt của bệnh nhân (Việt Nam - UTC+7).

Listing 2.1: Thuật toán Quét và Gửi nhắc nhở

```
Input: t_run (UTC timezone)
Output: Dispatch email reminders and store database
        notifications

t_local = t_run + 7 hours    // Convert to Asia/Ho_Chi_Minh
                             timezone

// Find pending/overdue medication schedules in the last 2
                             hours
```

```
Schedules = GetPendingSchedules(t_local, window = 2 hours)

FOR EACH schedule IN Schedules DO
    IF NOT HasNotificationSent(schedule.id) THEN
        // Trigger asynchronous email dispatch via worker
        CALL SendEmailAsync(
            to = schedule.user_email,
            subject = "Medication Reminder",
            body = FormatReminderBody(schedule.
                medication_name)
        )
        // Store notification for Realtime WebSocket push
        CALL CreateNotification(
            user_id = schedule.user_id,
            type = "prescription_reminder",
            message = "Time to take " + schedule.
                medication_name,
            reference_id = schedule.id
        )
    END IF
END FOR
```

2.4.2 Thuật toán đánh giá quyền truy cập thông tin bệnh nhân (Dynamic Authorizer)

Đảm bảo kiểm tra phân quyền chặt chẽ trước khi trả về bất kỳ chỉ số sức khỏe hoặc bệnh án nào của bệnh nhân cho bác sĩ yêu cầu.

Listing 2.2: Thuật toán Kiểm tra Quyền truy cập

```
Input: doctor_id, patient_id, requested_scope, current_time
Output: True (Allowed) or False (Denied)

// Doctor views their own patient data
IF doctor_id == patient_id THEN
    RETURN True
END IF

// Query valid permission from database
permission = SELECT id FROM consent_permissions
              WHERE patient_id = patient_id
                AND doctor_id = doctor_id
                AND requested_scope IN elements_of(scopes)
```

```

        AND granted_at <= current_time
        AND expires_at >= current_time
        AND revoked_at IS NULL
    LIMIT 1

IF permission is NOT NULL THEN
    // Log access to audit table
    INSERT INTO data_access_logs (action, user_id, table_name
        , record_id)
    VALUES ('read_patient_data', doctor_id, requested_scope,
        patient_id)
    RETURN True
ELSE
    RETURN False
END IF

```

2.4.3 Thuật toán truy xuất khẩn cấp qua mã QR (SOS Emergency Access)

Cho phép nhân viên y tế truy xuất ngay thông tin sinh mạng của bệnh nhân trong tình huống khẩn cấp mà không cần đăng nhập.

Listing 2.3: Thuật toán Truy xuất khẩn cấp

```

Input: qr_token, current_time, client_ip, client_ua
Output: SOSProfile or AccessError

token_record = SELECT * FROM emergency_tokens
                WHERE token = qr_token
                AND expires_at >= current_time
                AND deleted_at IS NULL
                LIMIT 1

IF token_record IS NULL THEN
    RAISE ERROR "Invalid or expired QR token"
END IF

patient_id = token_record.user_id
profile = SELECT blood_type, allergies, emergency_contact,
    privacy_settings
    FROM profiles WHERE id = patient_id LIMIT 1

// Filter fields based on user privacy settings
SOSProfile = EmptyObject()

```

```
IF profile.privacy_settings.show_blood_type == True THEN
    SOSProfile.blood_type = profile.blood_type
END IF
IF profile.privacy_settings.show_allergies == True THEN
    SOSProfile.allergies = profile.allergies
END IF
IF profile.privacy_settings.show_emergency_contact == True
    THEN
        SOSProfile.emergency_contact = profile.emergency_contact
END IF

// Log access history for patient audit
INSERT INTO emergency_access_logs (token_id, accessed_at,
    ip_address, user_agent)
VALUES (token_record.id, current_time, client_ip, client_ua)

RETURN SOSProfile
```

Chương 3

Thiết kế hệ thống

Chương này trình bày chi tiết về kiến trúc hệ thống **Medical Diary**, các công nghệ và công cụ được sử dụng ở từng tầng, cấu trúc thư mục chi tiết của mã nguồn Frontend và Backend, thiết kế phân tầng của các module chức năng, và cấu trúc triển khai hạ tầng thực tế.

3.1 Tổng quan Công cụ Sử dụng

Hệ thống được thiết kế theo kiến trúc phân rã thành ba tầng cốt lõi (Client - Server - Database) kết hợp với hạ tầng triển khai tự động (DevOps):

Tầng	Công cụ/Thư viện	Vai trò trong hệ thống
Frontend	React 18, TypeScript	Phát triển giao diện người dùng SPA (Single Page Application).
	Vite	Công cụ build mã nguồn nhanh và tối ưu hóa tài nguyên tĩnh.
	Tailwind CSS, Shadcn UI	Xây dựng giao diện Responsive, ứng dụng phong cách Soft UI hiện đại.
	Zustand	Quản lý trạng thái ứng dụng tập trung ở client.
	Axios	Client gửi yêu cầu HTTP kết hợp cơ chế tự động làm mới token truy cập.
Backend	FastAPI (Python 3.10)	Xây dựng RESTful API bất đồng bộ (Asynchronous HTTP Server).
	Pydantic	Thực hiện xác thực cấu trúc dữ liệu đầu vào và đầu ra (Data Validation).
	SQLAlchemy ORM	Tương tác với cơ sở dữ liệu qua các thực thể hướng đối tượng.
	Alembic	Quản lý lịch sử và chạy các bản di cư cơ sở dữ liệu (Database Migrations).
Database	PostgreSQL (Supabase)	Cơ sở dữ liệu quan hệ chính của hệ thống.
	Extension <code>pgcrypto</code>	Mã hóa bất đối xứng các thông tin nhạy cảm trực tiếp ở DB.
	Extension <code>pg_cron</code>	Lập lịch chạy các tác vụ nền định kỳ trên Database.
	Extension <code>pg_net</code>	Thực hiện các yêu cầu HTTP không đồng bộ từ Database đến Backend.
Deployment	Docker, Docker Compose	Container hóa mã nguồn Backend để tạo môi trường đồng nhất.
	Railway	Triển khai liên tục (CD) dịch vụ Backend từ kho lưu trữ GitHub.
	Vercel	Triển khai liên tục và lưu trữ mã nguồn tĩnh Frontend.

3.2 Thiết kế Chi tiết Frontend

Tầng Frontend chịu trách nhiệm tương tác trực tiếp với người dùng cuối, đảm bảo trải nghiệm Soft UI mượt mà, trực quan và bảo mật thông tin ở mức giao diện.

3.2.1 Cấu trúc thư mục Frontend

Mã nguồn Frontend của **Medical Diary** được tổ chức khoa học trong thư mục `frontend/src` nhằm đảm bảo khả năng bảo trì và mở rộng lâu dài:

- **api/**: Chứa các định nghĩa gọi API đến Backend. Đặc biệt là tệp cấu hình Axios Client tích hợp Interceptor để tự động bắt mã lỗi 401 và thực hiện cuộc gọi làm mới mã xác thực (refresh token) trước khi thử lại yêu cầu lỗi, giúp người dùng không bị gián đoạn trải nghiệm.
- **app/**: Quản lý cấu hình định tuyến (React Router Dom) và phân quyền truy cập trang (Private Routes cho Bác sĩ, Bệnh nhân, và Quản trị viên).
- **components/**: Chứa các thành phần UI dùng chung cho toàn bộ hệ thống (như `DataTable`, `StatCard`, các hộp thoại xác nhận, các thành phần biểu đồ, thanh điều hướng và thanh bên).
- **config/**: Chứa các cấu hình môi trường toàn cục (như API URL, Supabase Client initialization).
- **constants/**: Định nghĩa các hằng số hệ thống, danh mục các chỉ số sức khỏe, quyền hạn và trạng thái đơn thuốc.
- **hooks/**: Chứa các Custom Hooks phục vụ tái sử dụng logic xử lý (như `useAuth` quản lý phiên đăng nhập, `useHealthMetrics` xử lý dữ liệu biểu đồ).
- **pages/**: Tổ chức mã nguồn của các trang giao diện riêng biệt theo từng phân hệ chức năng (bao gồm `dashboard`, `auth`, `prescriptions`, `health-metrics`, `doctors`, `admin`, và `sos`).
- **store/**: Chứa các cửa hàng lưu trữ trạng thái toàn cục của Zustand (như `authStore` lưu thông tin phiên đăng nhập và `notificationStore` lưu thông báo thời gian thực).
- **styles/**: Lưu trữ tệp cấu hình CSS và định nghĩa biến giao diện sáng/tối (Dark Mode).
- **types/**: Định nghĩa các kiểu dữ liệu và Interface của TypeScript cho toàn hệ thống nhằm đảm bảo an toàn kiểu (Type Safety).

3.2.2 Luồng tương tác Frontend

Frontend giao tiếp với Backend thông qua các API bất đồng bộ. Khi người dùng thực hiện một hành động (ví dụ: đánh dấu thuốc đã uống):

1. Giao diện kích hoạt hành động tương ứng trong Zustand store.
2. Axios Client thực hiện gửi yêu cầu POST đến Backend với mã Token đính kèm ở Header.
3. Nếu nhận phản hồi thành công, giao diện sẽ cập nhật trạng thái hiển thị trực quan thông qua các hoạt ảnh nhẹ nhàng (Micro-animations).

3.3 Thiết kế Chi tiết Backend

Tầng Backend đóng vai trò bộ não điều khiển toàn bộ logic nghiệp vụ của hệ thống, xử lý các yêu cầu từ Client và đảm bảo tính toàn vẹn dữ liệu.

3.3.1 Cấu trúc thư mục Backend

Thư mục **backend/app** được thiết kế theo kiến trúc Module hóa (Modular Architecture), nơi mỗi chức năng nghiệp vụ độc lập được nhóm lại thành một thực thể riêng biệt:

- **core/**: Chứa các thiết lập nền tảng của hệ thống bao gồm cấu hình biến môi trường, thiết lập kết nối cơ sở dữ liệu bất đồng bộ (Database Session), và các tiện ích mã hóa mật khẩu.
- **middlewares/**: Chứa các bộ lọc trung gian xử lý yêu cầu trước khi đến Router (bao gồm CORS Middleware cấu hình tên miền được phép truy cập, Auth Middleware xác thực chữ ký JWT Token).
- **shared/**: Chứa các mã nguồn dùng chung giữa các module như các hàm định dạng ngày tháng, các mẫu ngoại lệ HTTP tùy chỉnh.
- **modules/**: Đây là lõi nghiệp vụ của hệ thống, chia nhỏ thành các thư mục con tương ứng với từng miền nghiệp vụ:
 - **auth/**: Đăng ký, đăng nhập, cấp phát và làm mới mã xác thực.
 - **users/**: Quản lý thông tin cá nhân và thiết lập quyền riêng tư.
 - **doctors/**: Quản lý hồ sơ bác sĩ và quy trình tải lên chứng chỉ hành nghề.
 - **prescriptions/**: Quản lý đơn thuốc và chi tiết các loại thuốc cần uống.
 - **diaries/**: Ghi chép nhật ký y tế hàng ngày.
 - **health_metrics/**: Lưu trữ và truy xuất các chỉ số sinh học tự động hoặc nhập tay.

- **consent/**: Quản lý các quyền ủy quyền chia sẻ dữ liệu y khoa từ bệnh nhân cho bác sĩ.
- **emergency/**: Cấp phát mã khẩn cấp cứu sinh (SOS Token) và truy xuất dữ liệu tối giản.
- **notifications/**: Quản lý thông báo và hỗ trợ đẩy thông báo đẩy thời gian thực.
- **admin/**: Cung cấp các API phê duyệt hồ sơ bác sĩ và theo dõi hoạt động toàn hệ thống.

3.3.2 Cấu trúc phân tầng trong từng Module

Để tuân thủ nguyên tắc tách biệt mối quan tâm (Separation of Concerns), mã nguồn trong mỗi module chức năng ở thư mục `modules/` luôn được chia nhỏ thành 4 tầng nhất quán:

1. **Tầng Mô hình dữ liệu (`models.py`)**: Sử dụng SQLAlchemy để định nghĩa ánh xạ cấu trúc bảng cơ sở dữ liệu quan hệ thành các lớp đối tượng Python. Đây là nguồn sự thật duy nhất cho schema dữ liệu của module.
2. **Tầng Xác thực dữ liệu (`schemas.py`)**: Sử dụng Pydantic để định nghĩa các cấu trúc dữ liệu đầu vào (Request) và đầu ra (Response). Tầng này tự động kiểm tra định dạng dữ liệu (như định dạng email, số điện thoại, ngày tháng) và trả về lỗi chi tiết trước khi dữ liệu đi vào tầng xử lý logic.
3. **Tầng Nghiệp vụ (`service.py`)**: Nơi chứa toàn bộ logic xử lý chính của bài toán. Tầng này trực tiếp tương tác với cơ sở dữ liệu thông qua cơ chế bất đồng bộ (AsyncSession), tính toán và trả kết quả về cho tầng điều khiển. Tầng nghiệp vụ không tương tác trực tiếp với giao thức mạng HTTP.
4. **Tầng Điều hướng và Điểm cuối (`router.py`)**: Định nghĩa các API endpoints của module, nhận các yêu cầu HTTP, kiểm tra quyền hạn của người dùng thông qua Dependency Injection của FastAPI, sau đó chuyển giao công việc cho tầng Nghiệp vụ xử lý và đóng gói dữ liệu phản hồi dựa trên Pydantic schemas.

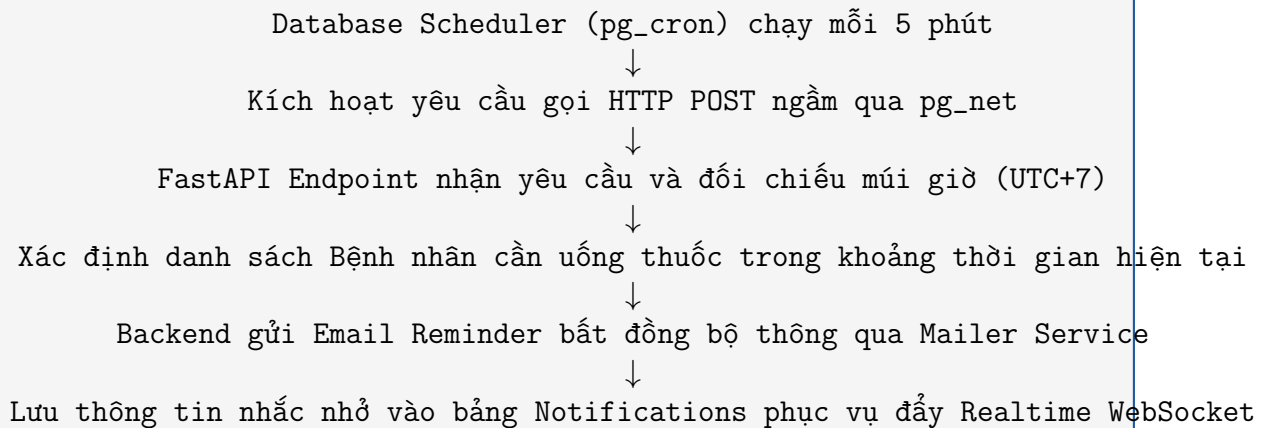
3.4 Thiết kế Cơ sở dữ liệu và Cơ chế Tự động hóa

Hệ thống sử dụng cơ sở dữ liệu PostgreSQL lưu trữ trên nền tảng đám mây Supabase để đảm bảo khả năng mở rộng và hiệu năng truy vấn cao.

3.4.1 Sơ đồ luồng hoạt động tự động hóa

Việc nhắc nhở uống thuốc và ghi nhận lịch sử sử dụng thuốc đòi hỏi tính tự động hóa cao, giải quyết bài toán đồng bộ múi giờ và giảm tải tài nguyên xử lý của Backend chính. Hệ thống thiết lập luồng vận hành tự động như sau:

Luồng tự động hóa nhắc nhở uống thuốc



Hình 3.1: Mô hình luồng tự động hóa tác vụ nhắc nhở y tế

3.5 Kiến trúc Triển khai (Deployment Architecture)

Để đảm bảo khả năng sẵn sàng phục vụ thực tế của hệ thống, các dịch vụ được triển khai thông qua hạ tầng đám mây tự động:

- **Đóng gói ứng dụng với Docker:** Toàn bộ mã nguồn Backend được bao bọc trong một Dockerfile tối giản hóa dựa trên hình ảnh gốc `python:3.10-slim`. Docker Compose được cấu hình để khởi tạo nhanh chóng toàn bộ cụm dịch vụ bao gồm FastAPI app và cơ sở dữ liệu PostgreSQL cục bộ cho mục đích phát triển và chạy thử nghiệm cục bộ (Staging/Local Development).
- **Triển khai Backend (Railway):** Dịch vụ Backend FastAPI được lưu trữ trên nền tảng Railway. Hệ thống liên kết trực tiếp với nhánh sản xuất (`main`) của kho GitHub. Khi có sự thay đổi mã nguồn, Railway tự động kích hoạt tiến trình dựng Docker Image, kiểm tra lỗi và triển khai phiên bản mới trực tuyến (Continuous Deployment) mà không gây gián đoạn dịch vụ.
- **Triển khai Frontend (Vercel):** Mã nguồn Frontend React được biên dịch thành các tệp tĩnh tối ưu hóa (HTML/CSS/JS) và phân phối trên nền tảng Vercel thông qua mạng lưới CDN toàn cầu. Tệp cấu hình `vercel.json` thiết lập luật điều hướng (routing rewrite) để đảm bảo các yêu cầu định tuyến của ứng dụng trang đơn (SPA) luôn được xử lý chuẩn xác bởi trình duyệt.

Chương 4

Thiết kế dữ liệu

Cơ sở dữ liệu sử dụng PostgreSQL với các bảng chính sau:

Bảng 4.1: *Bảng profiles*

Thuộc tính	Mô tả
id	Mã người dùng (khóa chính)
full_name	Họ và tên
email	Địa chỉ email
role	Vai trò trong hệ thống

Bảng 4.2: *Bảng medicines*

Thuộc tính	Mô tả
id	Mã thuốc (khóa chính)
name	Tên thuốc
description	Mô tả chi tiết

Bảng 4.3: *Bảng medication_schedules*

Thuộc tính	Mô tả
id	Mã lịch (khóa chính)
user_id	Khóa ngoại – người dùng
medicine_id	Khóa ngoại – thuốc
reminder_time	Thời gian nhắc nhở

Bảng 4.4: *Bảng medication_logs*

Thuộc tính	Mô tả
id	Mã nhật ký (khóa chính)
schedule_id	Khóa ngoại – lịch sử dụng
taken_at	Thời điểm xác nhận đã uống

Bảng 4.5: *Bảng emergency_contacts*

Thuộc tính	Mô tả
id	Mã liên hệ (khóa chính)
user_id	Khóa ngoại – chủ sở hữu
phone	Số điện thoại khẩn cấp

Chương 5

Kết quả đạt được

5.1 Các chức năng đã triển khai

- Đăng ký tài khoản.
- Đăng nhập hệ thống.
- Quản lý hồ sơ người dùng.
- Quản lý thuốc.
- Ghi nhận chỉ số sức khỏe và thông tin liên quan (nhịp tim, dị ứng, vaccine).
- Quản lý lịch nhắc uống thuốc.
- Theo dõi lịch sử sử dụng thuốc.
- Quản lý liên hệ khẩn cấp.
- Chức năng SOS.

5.2 Giao diện chính

- **Dashboard** – Tổng quan hệ thống.
- **Medicine Management** – Quản lý thuốc.
- **Reminder Management** – Quản lý lịch nhắc.
- **Medication Logs** – Nhật ký dùng thuốc.
- **Profile Management** – Quản lý hồ sơ.
- **SOS Page** – Trang khẩn cấp.

Chương 6

Đánh giá và hướng phát triển

6.1 Ưu điểm

- Hỗ trợ người dùng uống thuốc đúng giờ, cải thiện hiệu quả điều trị.
- Quản lý thông tin sức khỏe tập trung tại một nơi.
- Giao diện thân thiện, dễ sử dụng và dễ mở rộng.
- Tự động hóa toàn bộ quy trình nhắc nhở.
- Hệ thống phân tầng, dễ bảo trì và mở rộng.
- Hệ thống SOS hỗ trợ truy cập thông tin nhanh qua QR mà không cần đăng nhập.

6.2 Hạn chế

- Chưa tích hợp AI hỗ trợ phân tích sức khỏe và cảnh báo khi có dấu hiệu bất thường.
- Chưa có ứng dụng di động độc lập (native app).
- Chưa tích hợp thiết bị đeo thông minh (smartwatch) để thu thập thông tin.

6.3 Hướng phát triển

- Tích hợp AI phân tích lịch sử dùng thuốc và cảnh báo sớm.
- Tích hợp AI hỗ trợ phân tích điểm bất thường sức khỏe và hướng cải thiện.
- Xây dựng Mobile App hỗ trợ Push Notification theo thời gian thực
- Tích hợp smartwatch và các thiết bị đo khác để theo dõi sức khỏe liên tục.
- Phân tích dữ liệu sức khỏe nâng cao và lập báo cáo.
- Đồng bộ đa nền tảng (web, iOS, Android).

Chương 7

Kết luận

Medical Diary là hệ thống hỗ trợ quản lý nhật ký sử dụng thuốc và theo dõi sức khỏe cá nhân, được xây dựng dựa trên nền tảng tư duy tính toán. Trong quá trình phát triển đề tài, bốn thành phần cốt lõi đã được áp dụng đầy đủ:

1. **Phân rã vấn đề (Decomposition)** – Từ bài toán quản lý sức khỏe đã được chia thành các module độc lập, mỗi module chịu trách nhiệm một mảng chức năng riêng biệt giúp việc phát triển và bảo trì trở nên dễ dàng.
2. **Nhận dạng mẫu (Pattern Recognition)** – Nhận diện các mẫu lặp đi lặp lại trong thực tế đời sống (CRUD, Schedule,...) để tái sử dụng những mẫu thay vì thiết kế mới.
3. **Trừu tượng hóa (Abstraction)** – Chắt lọc các thuộc tính thiết yếu của từng đối tượng, loại bỏ thông tin thừa giúp giảm độ phức tạp và xây dựng mô hình dữ liệu phù hợp và đơn giản.
4. **Thiết kế thuật toán (Algorithm Design)** – xây dựng logic nhắc nhở và ghi nhận dùng thuốc một cách rõ ràng, hiệu quả.

Qua đó, đề tài đã minh họa thành công cách áp dụng tư duy tính toán để giải quyết một bài toán thực tiễn trong lĩnh vực chăm sóc sức khỏe, đồng thời tạo nền tảng vững chắc cho các hướng phát triển trong tương lai.